

Transmeta Code Morphing TM Software: Using Speculation, Recovery, and Adaptive Retranslation to Address Real-Life Challenges

Abstract

Transmeta's Crusoe microprocessor is a full, system-level implementation of the x86 architecture, comprising a native VLIW microprocessor with a software layer, the Code Morphing Software (CMS), that combines an interpreter, dynamic binary translator, optimizer, and runtime system. In its general structure, CMS resembles other binary translation systems described in the literature, but it is unique in several respects. The wide range of PC workloads that CMS must handle gracefully in real-life operation, plus the need for full system-level x86 compatibility, expose several issues that have received little or no attention in previous literature, such as exceptions and interrupts, frO, DMA, and self-modifying code. In this paper we discuss some of the challenges raised by these issues, and present the techniques developed in Crusoe and CMS to meet those challenges. The key to these solutions is the Crusoe paradigm of aggressive speculation, recovery to a consistent x86 state using unique hardware commit-and-rollback support, and adaptive retranslation when exceptions occur too often to be handled efficiently by interpretation.

1 Introduction

Transmeta's Crusoe® VLIW processor and CMS present an approach unique among commercial architectures: a microprocessor system with an internal VLIW instruction set architecture (ISA) with little resemblance to the external ISA (x86) that it presents to users. This approach allows a simple, compact, low-power microprocessor implementation, with the freedom to modify the internal ISA between generations, while supporting the broad range of legacy x86 software available. Producing robust runtime performance comparable to competing x86 implementations requires that CMS deal effectively with a number of difficult problems that have usually been ignored in the literature on binary translation and dynamic optimization. In this paper, we will sketch the structure of CMS, but our focus will be on several of the challenges it faced that set it apart from other systems described in the literature, and on the solutions we implemented. These challenges are natural consequences of CMS

objectives:

- CMS must faithfully implement the complete x86 architecture: all instructions (including memory-mapped I/O), architectural registers, and complete exception behavior.
- CMS can make no assumptions about the operating system running on the processor and cannot depend on information or other assistance from the system. It is a system-level implementation, not application-level, and even executes the BIOS code. One consequence is that it does not have access to the executable files of the applications it runs; all translation is done on-line as the target software executes.
- CMS must provide robust performance for a wide variety of systems and applications, ranging from games and media processing to desktop productivity and server applications. This requires dealing with unpleasant realities like self-modifying code and precise exceptions. It is important to note in this regard that CMS is not a migration tool; unlike past commercial systems, CMS is not an interim solution to be used during transition of the code base to a new architecture, and cannot deal with unusual but important performance problems by waiting for the code in question to be converted.

第二部分

提供了有關 Crusoe 處理器特徵和 CMS 結構的背景，以便進行後續討論。

第三部分

描述了 CMS 如何使用推測、恢復和自適應重翻譯來應對全系統、高性能動態二進制翻譯的多個挑戰。

第四部分

調查了相關工作。

2 Crusoe 和 CMS

Crusoe 處理器具有為簡單性而設計的微架構，將複雜但不頻繁的任務移入軟體中。雖然對架構的全面討論超出了本文的範疇，但我們在此提供一些與後續討論相關的細節。

Crusoe TM5800 是一個 VLIW 處理器。每條指令（稱為分子）可以向五個功能單元的子集發出兩個或四個類 RISC 操作（稱為原子）：兩個 ALU、一個記憶單元、一個浮點/媒體單元和一個分支單元。它擁有 64 個通用寄存器和 32 個浮點寄存器，允許將架構 x86 寄存器分配給專用的本地 VLIW 寄存器，並為 CMS 提供充足的可用集合。Transmeta VLIW 硬體幾乎沒有硬體互鎖。CMS 通過仔細調度來保證正確操作，必要時插入無操作指令。只有不可預測的長延遲操作，例如在快取中未命中的加載，其額外延遲由硬體自動處理。因為 CMS 可以根據處理器進行調整，未來的硬體世代可以改變操作延遲或本地 ISA 或微架構的其他方面，而不影響可見的 x86 架構。事實上，當前的 TM5000 家族與第一個 TM3000 家族 Crusoe 處理器有了顯著的演變，增加了原子以更有效地實現 x86 分段、16 位操作和間接分支，所有這些都不改變目標 ISA。下一代 Crusoe 處理器，TM8000 家族，將進一步進行本地 ISA 的變更，包括對指令格式的完全重新設計；這一切對在處理器上執行的 x86 代碼都是不可見的。

CMS 的結構類似於許多其他動態翻譯系統。最初，解釋器按順序解碼和執行 x86 指令，仔細注意記憶體訪問順序和故障的精確重現，同時收集執行頻率、分支方向和記憶體映射 I/O 操作的數據。當某段 x86 代碼的執行次數達到某個閾值時，其地址會傳遞給翻譯器。翻譯器選擇包括該地址的區域，生成本地代碼以實現所識別區域的 x86 代碼，並將翻譯及各種相關信息存儲在翻譯快取中。從那時起，直到某些東西使翻譯快取條目失效，CMS 在 x86 控制流到達翻譯的 x86 代碼區域時執行該翻譯。最初，翻譯分支的出口到查找例程（圖 1 中的“無鏈”路徑），該例程將控制轉移到下一個地址的現有翻譯或返回到解釋器。然而，一旦分支目標被識別為另一個翻譯，則分支操作會被修改為直接轉到那裡，這個過程稱為鏈接（Cmelik 等人）。因此，隨著時間的推移，經常執行的代碼區域開始完全在翻譯快取中執行，而不會有解釋、翻譯甚至分支目標查找的開銷。

II 解釋 II

下一步

圖 1：典型的 CMS 控制流

各種異常事件可能會中斷這一典型控制流。本文主要關注這些情況的處理，這些情況在圖 1 中由“故障”路徑表示。翻譯器是 CMS 中最大、最複雜的組件。它由解碼 x86 指令、選擇翻譯區域、分析該區域內的 x86 數據和控制流、生

成該區域的本地 VLIW 代碼、優化它和調度它的模塊組成。翻譯區域的選擇超出了本文的範疇，但它們可能相當大且複雜，包含長追蹤、IF 語句和嵌套循環。

包含最多 200 x86 指令。這提供了擴展的優化範圍。優化器對該區域執行多種傳統和 Crusoe 特定的優化，並將最終的本地 VLIW 代碼安排為一組單入口、多出口的追蹤。所有這些都是在密切關注成本的情況下完成的，因為翻譯器可能佔據執行時間的相當一部分。除了這些組件外，CMS 還包括一個運行時系統來處理設備、中斷和異常、電源管理以及翻譯緩存的垃圾回收。

3 推測、恢復和自適應重翻譯

CMS 需要提供一個完全忠實但高性能的遺留商業微處理器架構的實現，這提出了重大挑戰。一個關鍵的範式使我們能夠解決許多技術障礙，即推測、恢復和自適應重翻譯。在這個上下文中，推測是指在翻譯時對正在翻譯的代碼做出和利用未經證實的假設。（例如，翻譯器可能假設兩個特定的加載和存儲指令引用不重疊的內存。）這種類型的推測使得生成更高效的翻譯成為可能，但如果一個或多個假設被證明為錯誤，則可能產生不正確的結果。因此，這些假設必須在運行時以某種方式進行驗證，並在檢測到違規時採取適當的行動。CMS 使用硬件和軟件機制的組合來檢測失敗的假設。這些機制觸發本地異常，將控制權轉移到各種失敗模式的處理程序。CMS 對失敗的響應類似於它處理正常執行的方式。為了應對不頻繁的失敗，CMS 調用解釋器來處理該條件。解釋器雖然比執行翻譯慢得多，但實現了精確的 x86 語義，並在每個指令邊界保證正確的機器狀態。由於這種解決方案沒有前期的時間或空間成本，因此對於絕大多數翻譯來說效果非常好，這些翻譯在其生命週期內從未或很少失敗任何推測假設。然而，大多數推測類型在重複執行的翻譯中偶爾會失敗，在這種情況下，故障和解釋的方法會產生不可接受的開銷。為了優雅地應對這種情況，CMS 監控重複的失敗，並在認為失敗率過高時生成更保守的翻譯。為了減少保守翻譯的性能影響，CMS 還試圖將失敗的原因限制在比原始翻譯更小的區域的重翻譯中。Transmeta 本地 VLIW 處理器為各種推測和隨後的恢復提供硬件支持；我們在 3.1 小節中描述了這一機制。隨後的小節描述了 CMS 通過應用推測、恢復和自適應重翻譯的程序所面臨的挑戰：

- CMS 必須忠實地重現 x86 目標的精確異常行為，而不過度限制其翻譯的調度。

- **CMS** 必須在精確的 **x86** 指令邊界響應中斷，該系統擁有一致的目標狀態。
- **CMS** 必須高效處理內存映射 **I/O** 和其他系統級操作，而不懲罰正常（非 **I/O**）內存引用。
- 遺留 **PC** 軟件，特別是遊戲，通常包括性能關鍵的自修改代碼。類似的問題來自於包含代碼和數據的頁面，這在 **Windows/9X** 設備驅動程序、**BIOS** 和運行實時操作系統（如 **QNX**）的嵌入式系統中很常見。我們在本節中提供各種數據來說明這些挑戰。其中一些是在 **TM5800** 系統上收集的，但在大多數情況下，所需的數據無法輕易從硬件中提取，我們使用了來自 **Crusoe** 模擬器的數據，該模擬器提供準確的動態分子計數，但不具備周期精度。模擬基準集包括幾個 **Windows** 變體的啟動、**DOS**、**Linux** 和 **OS/2**，以及來自 **SPECcpu92** 和 **SPECint2000** 的基準測試、**Windows** 生產力應用程序和媒體應用程序（見附錄 **A** 的列表）。我們通常會從這個集合中呈現選定或摘要的數據。請注意，我們在本文中討論的所有問題都發生在應用程序中，儘管某些問題（例如，內存映射 **I/O**）在系統代碼中更為常見。

3.1 硬件支持推測和恢復

編譯器通常通過生成補償代碼來處理推測的恢復，該代碼重新執行錯誤排序的操作，執行從推測代碼路徑中省略的操作，並糾正寄存器分配中的不匹配

（**Freudenberger** 等人）。使用這種方法，需要硬件支持來延遲潛在故障指令移動到分支上方的故障（例如，**Smith** 等人），檢測按順序調度的重疊內存操作，並分支到 17。

補償碼（例如，記憶體衝突緩衝區，**Gallagher** 等人，或 **Intel IA-64 ALAT**）。相對而言，**Crusoe** 原生 **VLIW** 處理器提供了一種優雅的硬體解決方案，支持任意類型的推測和隨後的恢復，並與代碼變形軟體緊密合作。所有持有 **x86** 狀態的寄存器都是影子寄存器；也就是說，每個寄存器存在兩個副本，一個是工作副本，另一個是影子副本。正常原子僅更新寄存器的工作副本。如果執行到達翻譯的結尾，特殊的提交操作會將所有工作寄存器複製到其相應的影子寄存器中，提交翻譯中所做的工作。另一方面，如果在翻譯內部發生任何異常情況，例如 **CMS** 的翻譯假設之一失敗，運行時系統會通過回滾操作撤銷自上次提交以來執行的所有分子效果，該操作將影子寄存器的值（在前一次翻譯結束時提交）複製回工作寄存器。回滾後，**CMS** 通常會解釋與故障翻譯相對應的 **x86** 指令，按照原始程序順序執行它們，處理遇到的任何特殊情況，並在必要

時調用 **x86** 異常處理程序。提交和回滾也適用於記憶體操作。存儲數據保存在一個門控存儲緩衝區中，只有在提交時才會釋放到記憶體系統。在回滾時，尚未提交的存儲可以簡單地從存儲緩衝區中刪除。為了加速沒有回滾的常見情況，該機制的設計使得提交操作實際上是“免費”的，而回滾原子的成本低於幾次分支錯誤預測。在接下來的子部分中，我們描述 **CMS** 利用提交/回滾機制的幾種方式。

3.2 精確異常

在沒有特殊硬體支持的情況下，對於在靜態調度主機上的動態翻譯系統來說，正確建模目標 **ISA** 的異常語義同時實現高性能是困難的，甚至是不可能的。主要原因是異常語義對指令調度施加了嚴格的約束。在 **x86 ISA** 中，異常是精確的：當一條指令引發異常時，所有在其之前的指令必須完成，然後才能報告異常，且後續的指令不得完成。沒有特殊硬體支持的情況下解決這個問題會過度約束主機指令的調度，和/或需要補償碼，這兩者都可能在沒有異常發生的常見情況下降低性能。但是，隨著對提交和回滾的硬體支持以及基於解釋器的恢復程序的到位，**CMS** 在調度翻譯指令方面具有更大的靈活性。它可以重新排序潛在故障的原子或將它們提升到條件分支之上，而不需要傳統控制推測所需的記帳，並且不會生成佔用空間的補償碼。然而，這種方法的後果是，對於應在 **x86** 層面反映的故障，**CMS** 必須確定故障是真實的還是由推測性指令重新排序造成的。如果解釋器在不遇到故障的情況下重新執行整個翻譯的指令，那麼它就是推測性的，如果它不頻繁，**CMS** 可以忽略它並繼續正常執行。處理重複故障的首選策略取決於其類別。對於真正的 **x86** 故障，我們在故障指令周圍縮小翻譯大小。這減少了必須回滾和/或解釋的工作量，因為相鄰區域可以保持大且高度優化。我們最終可以運行所有但故障指令的翻譯，這成為一個零指令翻譯，僅調用解釋器來執行故障指令。對於經常重複的推測性故障，我們以更保守的政策重新翻譯，這些政策可能會消除遇到的故障類型，首先將翻譯切割成更小的區域，以便其中大部分仍然可以受益於激進的翻譯。新的翻譯會跟蹤所使用的政策，因此如果出現需要不同保守政策的其他問題，**CMS** 將把它們添加到現有的政策中，以避免在無法比較的政策之間來回跳躍，而這兩者都無法解決兩個問題。

3.3 中斷

提交和回滾在中斷方面具有類似的目的。因為中斷會導致回滾到一致的目標狀態，翻譯代碼不必擔心在指令之間與 **x86** 狀態不一致的中間狀態中的中斷。中

斷不會觸發自適應重新翻譯，因為它們通常與執行時的翻譯無直接關係。提交和回滾也可以等同於檢查點和重啟。

3.4 記憶體映射 I/O

克魯索系統（處理器和 CMS）旨在透明地運行為 x86 架構編寫的任意代碼，包括操作系統和應用程序軟件。除了準確實現 x86 系統級架構的許多邊界情況的明顯困難外，CMS 還必須正確實現與物理設備的低級 I/O 交互。與 I/O 交易相關的最重要規則之一是，它們必須按照原始（x86）程序順序執行，因為它們會觸發與外部設備的不可逆交互。在 x86 架構中，設備可以通過兩種不同的機制訪問：顯式 I/O 指令（“in/out”）和記憶體映射訪問。前者易於識別並適當翻譯。然而，記憶體映射 I/O 在翻譯時無法與常規記憶體訪問區分。此外，給定的 x86 指令可以在程序執行過程中訪問常規記憶體和 I/O 空間。

圖 2：抑制記憶體重排序造成的降級

DOS 啟動 Linux 啟動 OS/2 啟動 Windows 95 啟動 Windows 98 啟動 Windows ME 啟動 Windows NT 啟動 Windows XP 啟動平均（所有啟動）Quake Demo2
（DOS）023.eqntott（Linux）026.compress（Linux）072.sc（Linux）085.gcc
（Linux）047.tomcatv（Linux）048.ora（Linux）052alvinn（Linux）
077.mdljsp2（Linux）多媒體（Win98）CPUmark99（Win98）Quattro Pro
（WinNT）Wordperfect（WinNT）平均（所有應用程序）

完全抑制記憶體重排序以解決此問題將是一個嚴重的障礙。為了說明這一點，我們運行了帶有和不帶有記憶體操作重排序的基準套件的模擬。上面的圖 2 展示了一組代表性的結果，以及整個集合的平均值（見附錄 A）。幾個啟動的降級低於 5%，但在 Windows/ME 啟動中成本高達 26%。應用程序的降級則大得多。為了解決這個問題，克魯索硬體上的加載和存儲原子指定它們是否已經相對於原始 x86 程序進行了重排序。當這樣的推測性記憶體原子訪問映射到 I/O 空間的記憶體頁面時，硬體會引發異常。在這一點上，CMS 執行回滾到先前提交的狀態並進行解釋。如果故障過於頻繁，CMS 會重新生成翻譯，這次不重排序有問題的記憶體引用。

3.5 數據推測

即使對於訪問記憶體而非設備的記憶體操作，翻譯器也常常無法證明加載和存儲地址不重疊；這也排除了重排序。一個關鍵的見解是，在實踐中，如果重疊

不明顯，記憶體引用很少重疊，因此重排序通常是安全的（並且有利的）。克魯索提供了簡單的硬體支持（別名硬體），允許 CMS 重排序選定的記憶體引用，硬體在運行時承擔驗證重排序引用確實不重疊的負擔。如果硬體檢測到違規，它會引發異常，CMS 可能會在解釋器中調用回滾和保守的重新執行來處理該條件。

圖 3：因沒有別名硬體造成的降級

0%10%20%30%40%50%DOS 啟動 Linux 啟動 OS/2 啟動 Windows 95 啟動
Windows 98 啟動 Windows ME 啟動 Windows NT 啟動 Windows XP 啟動平均
（所有啟動）Quake Demo2（DOS）023.eqntott（Linux）026.compress
（Linux）072.sc（Linux）085.gcc（Linux）047.tomcatv（Linux）048.ora
（Linux）052.alvinn（Linux）077.mdljsp2（Linux）多媒體（Win98）
CPUmark99（Win98）Quattro Pro（WinNT）Wordperfect（WinNT）平均（所有應用程序）

模擬數據來自我們的基準套件，顯示了別名硬體的性能優勢。上面的圖 3 展示了不使用別名硬體所導致的性能下降，這幾乎和完全不重新排序一樣嚴重。Crusoe 的別名硬體比在亂序處理器中強制執行內存約束所需的硬體簡單得多。它也比其他為 VLIW 處理器建議的方法更簡單，例如內存衝突緩衝區或 IA-64 ALAT。這些方法使用完全關聯的表格，並配備硬體機制來確定要覆蓋的受保護地址表條目，並檢查每個亂序內存操作，而 Crusoe 則要求翻譯器明確指定這一點。重複故障通過將故障翻譯切割成更小的區域來處理，並通過調度任何仍然故障的區域而不進行推測性加載/存儲重新排序。

3.6 自我修改代碼 (SMC)在執行過程中修改自己的程序可能會對任何微架構造成重大問題。對於 CMS 來說，這表現在保持翻譯緩存與其相關的 x86 代碼一致的問題上。CMS 再次進行推測，這次推測它翻譯的 x86 代碼不會改變。在這種情況下，問題的檢測和適應技術都是值得關注的。Transmeta 最初檢測 SMC 的方法是每當 CMS 翻譯該頁上的 x86 代碼片段時，簡單地對該 x86 內存頁進行寫保護。如果該受保護頁上的數據後來被修改，無論是由程序明確地還是由系統頁面活動隱式地修改，則會發生故障，CMS 將丟棄受影響的翻譯。頁級保護對於正確性是足夠的，並且在常見情況下對性能至關重要，但它並未有效處理自我修改代碼。如果在性能關鍵的循環中發生代碼和可寫數據共享，它也無法很好地處理。儘管這些在現代編譯應用中變得不那麼常見，但設備驅動程序、像 Quake 這樣的遊戲、嵌入式代碼等，使用了將代碼和本地（靜態）數據混合的

技術。這在 BIOS 和嵌入式軟件中尤其常見，這些軟件受到空間限制，並且通常廣泛使用組合代碼。SMC 會產生兩種成本。第一種是在頁面寫入時處理故障並使與該頁面相關的翻譯失效。第二種是在下一次執行該頁面上的代碼時重新生成翻譯。以下小節討論了三種技術，以最小化檢測到包含混合代碼和數據的頁面寫入的成本，然後是兩種適應實際自我修改代碼的技術。

3.6.1 精細粒度保護 Crusoe 處理器提供對內存進行寫保護的硬體支持，粒度比完整頁面更細。關鍵的見解是，只有在同一時間需要對少數幾個頁面進行更細的粒度（例如，幾個共享可寫代碼和數據的頁面）。因此，只有少數幾個頁面需要在硬體緩存中擁有精細粒度條目，並且軟體故障處理程序可以在未命中時從內存更新緩存，從而允許一個小而簡單的硬體結構。所支持的粒度並不總是能夠識別受影響的單個翻譯，但通常將影響範圍縮小到幾個，減少每個故障的數量和失效翻譯的數量。為了避免對虛擬內存分頁的常見情況進行過度處理，對受保護頁面的 DMA 寫入會使該頁面的所有翻譯失效。我們模擬了幾個基準，以展示精細粒度保護的好處，並比較了有無精細粒度特徵的保護故障數量，以及每個 x86 指令執行的分子總體減速。結果見下表 1。"故障"列給出了沒有精細粒度支持的保護故障數量與有精細粒度支持的數量的比率，"減速"列顯示了因此對每個 x86 指令執行的分子影響。

表 1：沒有精細粒度保護的減速故障 減速 Win95 啟動 52.8x 2.2xWin98 啟動 59.4x 3.8xMultimediaMark 46.8x 1.6xWinStone Corel 54.2x 2.1xQuake Demo2 7.7x 1.02x

3.6.2 自我重新驗證翻譯如果 CMS 確定翻譯因為數據存儲在與代碼相同的區域而遇到合法的保護故障，它可以通過添加一個前導來降低故障的成本，前導是一段在進入翻譯之前調用的代碼段。前導通常用於臨時監控目的，並允許在不干擾翻譯的情況下輕鬆安裝和移除。插入前導涉及移除任何

現有鏈接到翻譯，並用序言地址替換其起始地址。一旦識別出自我重新驗證的候選翻譯，則將其標記。下次遇到時，將重新翻譯以捕獲翻譯的 x86 代碼（最初不會保留）。稍後，如果細粒度保護故障的處理程序確定翻譯可能受到影響，則啟用序言並關閉保護，以避免再次故障的成本。當下次調用翻譯時，序言驗證與翻譯相對應的 x86 代碼未發生變化，重新啟用保護，重新驗證 x86 代碼，禁用序言，然後執行翻譯。這種技術並不消除由寫入引起的保護故障。但它在對受保護區域的每次寫入最多執行一次故障處理程序和檢查，在對翻譯的每次執行中最多執行一次，因此如果寫入的頻率遠低於受影響翻譯的執行頻

率，則可以非常高效。在重新翻譯以捕獲要檢查的 x86 代碼後，翻譯以正常優化速度執行，除非對受保護區域有寫入。作為好處的例子，Quake Demo2 基準測試在自我重新驗證下的幀率比沒有自我重新驗證高出 28%。如果保護故障確實經常發生，則故障處理程序和檢查的開銷是顯著的，因為重新驗證的成本可能至少與執行翻譯一樣昂貴。此外，如果是翻譯本身在其相關的 x86 區域上寫入，則此技術無法工作，因為寫入發生在檢查序言完成之後，導致新的故障並阻止前進。因此，對於這種情況，以下優化故障檢測的技術可能更有效。

3.6.3 自我檢查翻譯

與其在創建翻譯時保護 x86 頁面，不如將內存頁面保持未保護，並讓翻譯本身檢查源 x86 字節是否未更改，通過提取它們並將其與創建翻譯時的值進行比較。我們可以將檢查代碼合併到正常的翻譯代碼中，因為如果檢測到不匹配，我們可以回滾已經發生的任何翻譯效果。插入的檢查代碼必須遵循調度約束。檢查 x86 操作的提取必須在邏輯上出現在任何存儲之後，包括正在檢查的操作，並且在與正在檢查的操作相同的控制流路徑上。然而，使用別名硬件，檢查的提取可以相對於存儲進行重排序，如第 3.5 節所述。因此，自我檢查翻譯一次的開銷比自我重新驗證一次的開銷小得多，儘管如果翻譯在保護故障之間執行多次，其平均成本可能會高得多。為了評估自我檢查翻譯的典型成本，我們正常運行了基準套件的模擬，並強制所有翻譯進行自我檢查。自我檢查使代碼大小平均增加 83%（範圍從 58% 到 100%），執行的分子平均增加 51%（範圍從 11% 到 124%）；由於緩存效應，實際運行時影響會更高。儘管自我檢查翻譯的成本低於解釋或重新翻譯，但從上述數據中可以看出，它們的開銷仍然顯著，特別是對於長翻譯（在絕對成本上）。因此，即使我們可以使用這種技術來消除不必要的自我修改失敗，我們首先嘗試通過生成較小的翻譯來適應，以便檢查的代碼最少。

3.6.4 風格化 SMC

上述技術僅在代碼實際未更改的情況下有用，即如果保護故障是由於在與代碼相同的頁面上寫入數據而導致的。這裡描述的最後兩種技術是適應真正自我修改代碼的方法。許多依賴自我修改代碼的 PC 應用程序以非常風格化的方式進行。舉例來說，一種常見的方法是在進入內部循環之前修改內部循環中指令的立即數或偏移量字段。可以通過以這種方式翻譯原始 x86 代碼來避免在這種特殊情況下不斷重新翻譯，這樣翻譯在運行時從代碼流中加載相關的立即數字段。考慮 x86 指令：

這可以翻譯成 Crusoe 代碼：

這種翻譯是有效的，無論應用程序如何修改 "0x123456" 立即數字段。請注意，這種技術必須與自我檢查或自我重新驗證結合使用，以驗證除立即操作數之外的指令字段未被修改。這種方法似乎在寄存器稀缺的機器（如 x86）上特別受歡迎，當存在循環不變常數但沒有寄存器可以在整個循環中保存它們時。例如，遊戲 Doom 在關鍵循環中使用它，並且它也出現在當前應用程序中，如 Adobe Premiere。然而，它在可移植的編譯基準（如 SPEC）中很少出現。

3.6.5 翻譯群組

有時，自我修改的代碼會重複寫入並執行少數幾個版本的重寫 x86 代碼。例如，Windows/9X 中的設備無關 BLT 驅動程式在我們檢查的基準中使用了多達 33 個版本，具體版本取決於要執行的操作和圖形芯片的硬體能力。在這種情況下，當舊版本重新出現時，擁有舊翻譯是可取的。CMS 將這些翻譯保存在翻譯群組中。這些是相同 x86 代碼區域的翻譯列表，當前活動的翻譯位於列表的最前面。如果第一個翻譯在保護故障後未能通過自我檢查，則會檢查其他翻譯是否與 x86 代碼當前匹配，然後再生成新的翻譯，任何找到的匹配翻譯將成為當前翻譯。由於這些技術，在實際遇到的情況中，CMS 穩健地為自我修改代碼和混合代碼與數據獲得良好的性能。

4 相關工作

CMS 與模擬、二進制翻譯和動態優化文獻最為相關，這些文獻有著悠久的歷史。在接下來的評論中，我們專注於軟體模擬系統，儘管有些可能具有促進模擬的硬體特徵。我們將軟體模擬系統分類為解釋器（逐條指令，無記憶體）、靜態翻譯器（離線）和動態翻譯器（在線）。（這是 Altman 等人的分類，他們使用“模擬器”而不是“解釋器”。）CMS 包含了解釋器和動態翻譯器（我們簡稱為翻譯器）。我們稱模擬的架構為目標，稱模擬的架構為主機。許多模擬系統是自我主機的，即主機和目標架構是相同的。這類系統通常是為了優化或儀器化的目的而創建的。一個著名的近期動態優化系統是來自 HP 實驗室的 Dynamo（Bala 等人）及其後繼者 DELI。Dynamo 的高級架構與 CMS 相似，但它可以回退到高效的本地執行，因此不需要對有問題的代碼進行翻譯，或者根本無法改進。出於這個原因，自我主機系統的權衡與像 CMS 這樣的系統有著相當不同的特點。另一個豐富的研究領域是虛擬目標模擬器，其中目標架構是一個專門設計的虛擬機，而不是物理架構。一個有趣的例子來自 IBM 將 AS/400

系統遷移到 PowerPC 架構，這是基於 AS/400 編譯器在應用程序對象代碼中包含的抽象機器代碼的靜態翻譯 (Soltis)。Java 虛擬機是一個更為人知的例子。它們模擬了一種專門設計的抽象字節碼，旨在能夠在各種機器上高效解釋。從早期的僅解釋器系統，這些模擬器已發展成為複雜的動態優化器，如 Sun 的 HotSpot、IBM 的 Jalapeño (Burke 等人) 和 LaTTe (Yang 等人)。這些系統與 CMS 有很多共同之處，包括翻譯成本和代碼質量之間的權衡。但虛擬機語義受到嚴格控制，避免了我們在本文中討論的大多數問題。跨主機模擬器在不同主機上模擬目標架構，必須更全面地處理各種目標代碼。一個常見的目的就是將代碼從目標架構移動到主機架構，通常是為了促進客戶從舊架構遷移到旨在取代它的新架構。例子包括 DEC 從 DEC VAX/VMS 到 Alpha/OpenVMS (VEST) 的遷移工具，以及 MIPS/Ultrix 到 Alpha/OSF1 (mx, 見)。這些系統不希望出現性能下降，但它們通常受益於比目標快得多的主機，並且問題代碼可以移植到新架構的本地代碼上。例如，VEST 項目的目標明確允許拒絕某些目標代碼，並提供診斷信息以指導手動干預。稍後的 Alpha 遷移項目是 FX!32，用於在 Windows NT Alpha 上運行 Windows NT x86 代碼 (Chernoff 等人)。FX!32 使用一個解釋器和一個靜態翻譯器，該翻譯器由解釋目標代碼觸發，但離線運行並將其翻譯保存在數據庫中。它不是 x86 的完美模擬器，例如執行 64 位而不是 80 位浮點，並且不支持 Windows NT 調試 API，因為它無法在任意點重新生成 x86 狀態。最近的一個商業項目是 HP 的 Aries，用於從 HP-PA 遷移到 IA-64 (Zhang 等人)。它的解釋器和動態翻譯器架構更類似於 CMS。然而，它僅執行單塊翻譯，並且像在內存中保持目標浮點寄存器影像這樣的弱點可能會在重要類別的代碼上造成嚴重的性能問題。參考資料未提供任何性能信息，據我們所知，該項目尚未完成。另一類遷移工具是由一個供應商設計的，旨在捕獲為另一個供應商架構創建的應用程序。早期的一個例子是 Hunter System 的 XDOS x86 DOS 模擬器，這是一個靜態翻譯器。模擬的軟體並不打算成為目標系統的主要用途，因此性能要求較為適中。困難的應用程序可能 22。

被特殊案例修改處理，翻譯通常需要顯著的手動干預。所有上述系統都有 CMS 無法使用的逃生閥，CMS 必須無縫執行任何 x86 軟體，並且必須提供與持續改進的硬體專用 x86 微處理器競爭的性能。最重要的區別是它們都是不處理系統代碼的應用層模擬器，而是將系統調用重定向到主機上的類似系統調用。然而，CMS 不需要模擬設備行為，因為其主機 I/O 子系統與目標相同。與 CMS 目標和約束最接近的系統是 IBM Research 的 DAISY (Ebcioğlu 等人)。DAISY

是 PowerPC 或 System/390 目標在樹狀 VLIW 主機上的全系統實現，具有類似於 CMS 的解釋器和動態翻譯器架構。其翻譯區域選擇不同（樹狀區域與 CMS 中的更一般代碼段），它使用狀態修復方法來處理精確異常（Gschwind 等人），而不是 CMS 的提交和回滾方法，並且僅使用細粒度保護方法來處理自修改代碼。參考文獻未討論我們的其他挑戰。還有許多其他二進制翻譯系統。更廣泛的先前工作討論可以在 Altman 等人中找到。

結論

CMS 是一個商業可用的系統，提供高性能、完全兼容的 x86 ISA 在不同主機 ISA（Crusoe 原生 VLIW）上的實現。CMS 在整體架構上與文獻中描述的多個二進制翻譯系統相似，但其成功的關鍵在於對本文中描述的挑戰的關注。積極推測的範式，在檢測到異常時回滾到一致狀態以進行恢復，並自適應地重新翻譯以處理重複異常，是 CMS 解決方案的一個強大部分。這些挑戰在嘗試運行各種日常工作負載之前不會顯現出來，但正是處理這些挑戰使 CMS 成為一個穩健的產品，而不是一個實驗系統。